

Welcome to **DeepWonder3D**, a comprehensive computational framework designed for the rapid and robust extraction of neuronal information from 3D calcium imaging datasets. This pipeline orchestrates the transition from raw, noisy volumetric data to precise 3D spatial localization and high-fidelity temporal traces, facilitating the analysis of large-scale neuronal dynamics.

Quick Start

After cloning the whole repository, only three commands are required to run the DeepWonder3D demo.

```
conda activate dw3d
cd DeepWonder3D_pytorch
python main_pipeline_3d.py
```

Table of Contents

- [I. Preparation](#)
 - [1. Pre-trained weights and calibration \(/pth \)](#)
 - [2. Input datasets \(/datasets \)](#)
 - [3. PSF fitting and 3D localization](#)
 - [PSF parameters](#)
 - [Structure of psffit_matrix](#)
- [II. Parameter Configuration](#)
- [III. Modular Pipeline Architecture](#)
- [IV. Pipeline Execution and Output](#)
 - [1. 2D vs. 3D processing](#)
 - [2. Hierarchical result structure](#)
- [V. Model Training \(main_train.py \)](#)

I. Preparation

Before executing the pipelines, ensure your local directory is structured as follows to allow for seamless data ingestion and model initialization:

1. Pre-trained weights and calibration (/pth)

The `/pth` directory must contain the necessary network weights and the optical calibration matrix:

- **Model weights:** `DENO_pth`, `RMBG_pth`, `SEG_pth`, and `SR_pth`.
- **Calibration:** `psffit_matrix.mat` (essential for 3D localization).

Two pre-trained models are provided in `DENO_pth`, `RMBG_pth`, `SEG_pth`, and `SR_pth`, optimized for data with pixel sizes of 1 μm and 2 μm , respectively.

2. Input datasets (/datasets)

Raw imaging data should be organized within the `datasets/` directory. For standard testing, place your volumetric files in `datasets/test/`.

In this repository, demo data is provided in `./datasets/test/`, along with the corresponding `./pth/psffit_matrix.mat` (see [3. PSF fitting and 3D localization](#) for details).

3. PSF fitting and 3D localization

In the final step of **3D localization**, the pre-computed `./pth/psffit_matrix.mat` file will be used.

PSF parameters

The default PSF parameters are as follows:

- **Number of views:** A total of 15×15 views, but only 81 views within the central circular region ($r = 5$) are used.
- **Experimental settings:**
 - Magnification: 10×
 - Excitation wavelength: 525 nm
 - Axial range: $-150 \mu\text{m}$ to $+150 \mu\text{m}$
 - z-step size: $3 \mu\text{m}$
 - Numerical aperture (NA): 0.4
 - Objective focal length (fml): $1251 \mu\text{m}$
 - Tube lens focal length (ftl): 180 mm

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70	71	72	73	74	75
76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100	101	102	103	104	105
106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
121	122	123	124	125	126	127	128	129	130	131	132	133	134	135
136	137	138	139	140	141	142	143	144	145	146	147	148	149	150
151	152	153	154	155	156	157	158	159	160	161	162	163	164	165
166	167	168	169	170	171	172	173	174	175	176	177	178	179	180
181	182	183	184	185	186	187	188	189	190	191	192	193	194	195
196	197	198	199	200	201	202	203	204	205	206	207	208	209	210
211	212	213	214	215	216	217	218	219	220	221	222	223	224	225

Structure of `psffit_matrix`

The matrix stored in `psffit_matrix.mat` contains the following columns:

Column	Description
1	Source view ID
2	Target view ID
3	Linear coefficient of lateral shift in the x direction (α_x , unit: shift/z)
4	Intercept of x-direction shift
5	Linear coefficient of lateral shift in the y direction (α_y , unit: shift/z)
6	Intercept of y-direction shift

⚠ IMPORTANT NOTE!!!

Since the PSF obtained under your experimental conditions may differ from ours, **before running** `main_pipeline.py` **on your dataset**, please first execute `get_PSFfit_matrix.py` in this directory to generate a `psffit_matrix.mat` that is compatible with your data.

II. Parameter Configuration

The pipeline's behavior is governed by several key parameters defined in the `__main__` block of the execution scripts:

Parameter	Meaning	Note
<code>now_pixel_size</code>	The spatial resolution of input raw data (μm).	Derived from microscope metadata.
<code>target_pixel_size</code>	The desired spatial resolution after DW3D enhancement (μm).	This repository provides pre-trained model weights (<code>.pth</code>) for both 1 μm and 2 μm resolutions for user selection.
<code>t_resolution</code>	The temporal resolution (i.e., frame rate) of input raw data (Hz).	Derived from microscope metadata.
<code>GPU_index</code>	Specifies which GPU device to use.	e.g., <code>'0'</code> or <code>'0,1'</code> .
<code>type</code>	Selector string for active pipeline modules.	e.g., <code>'deno_sr_rmbg_seg_mn_vm'</code> .

While the key parameters above require manual definition in `main_pipeline_2d.py` or `main_pipeline_3d.py`, other auxiliary and fine-grained configurations are predefined in `para_dict.py`. Advanced users may modify that file to adjust internal model hyperparameters; however, please note that suboptimal or excessively aggressive hyperparameter tuning may adversely impact DW3D's performance. Should you choose to explore these advanced configurations, please refer to the comprehensive comments within the source code for more guidance.

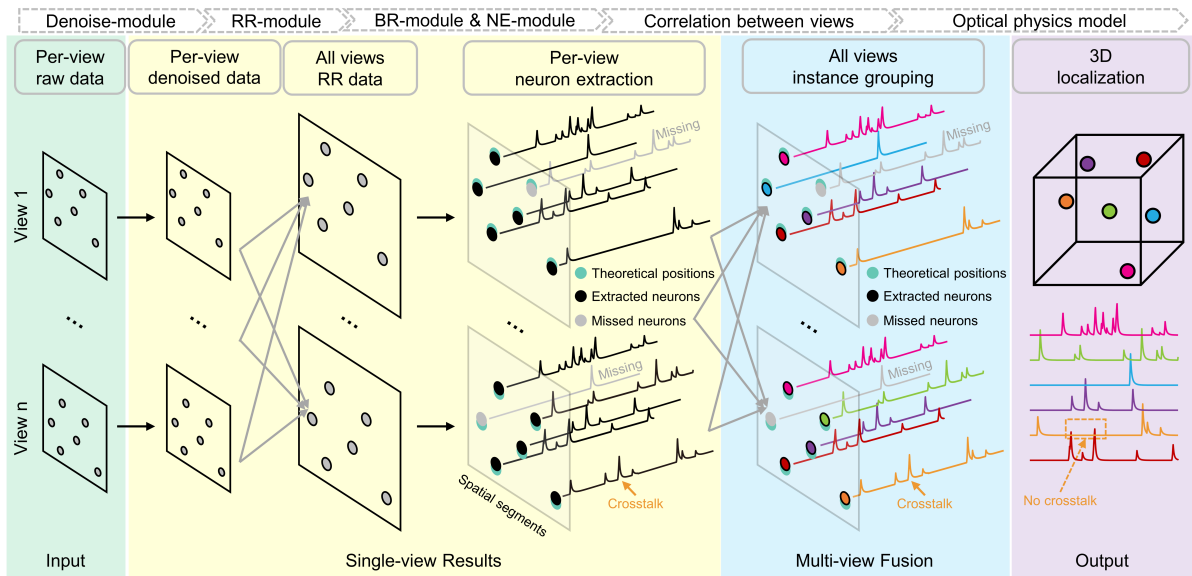
III. Modular Pipeline Architecture

DeepWonder3D utilizes a modular sequence of processing steps to ensure robustness, fidelity, and computational efficiency. Users can configure the execution via the `type` parameter to include specific steps:

- **DENO (Denoising)**: Enhances the signal-to-noise ratio (SNR) of raw data.
- **TR (Temporal Resolution)**: Performs temporal resampling to adjust the frame rate to the required resolution.
- **SR (Spatial Resolution)**: Applies spatial resampling to achieve target spatial resolution.
- **RMBG (Remove Background)**: Removes background fluorescence to isolate calcium transients.
- **SEG (Segmentation)**: Generates short-term spatial semantic masks of neuron candidates.
- **MN (Merge Neurons)**: Finalizes instance segmentation (i.e., merges neuron instances via spatiotemporal connectivity analysis and extracts their corresponding temporal traces).

- **VM (View Merging): (3D Only)** Executes multi-view fusion and 3D localization utilizing Point Spread Function (PSF).

Code implementation (step)	Manuscript reference (module)
DENO	Denoising Module
TR & SR	Resolution Registration Module (RR-module)
RMBG	Background Removal Module (BR-module)
SEG & MN	Neuronal Extraction Module (NE-module)
VM	Multi-view Fusion Module (MVF-module)



IV. Pipeline Execution and Output

1. 2D vs. 3D processing

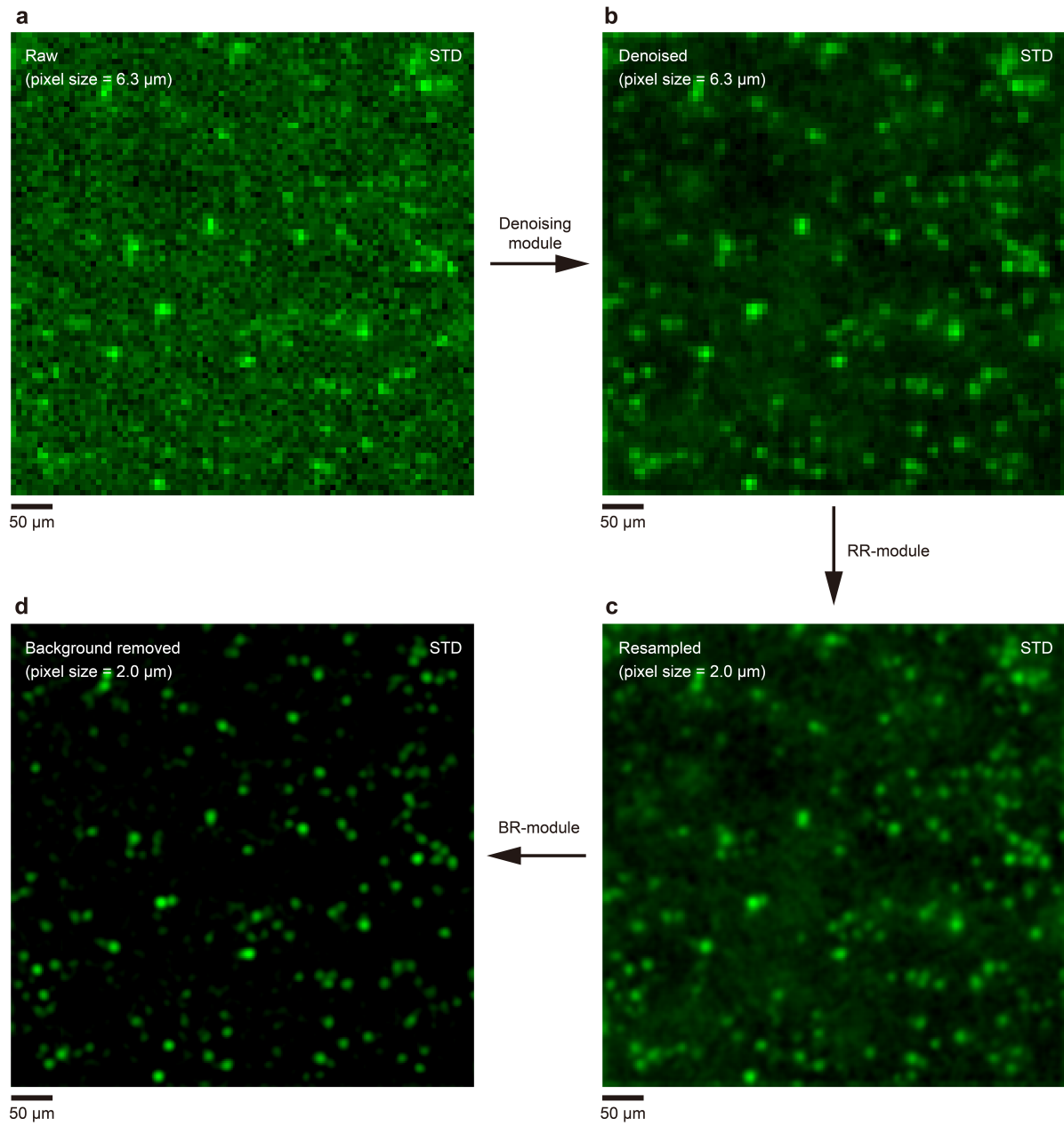
- `main_pipeline_2d.py`: Optimized for single-plane or maximum intensity projection (MIP) data where axial depth is not required.
- `main_pipeline_3d.py`: Integrates **View Merging (VM)** to achieve multi-view fusion and 3D localization of neurons.

2. Hierarchical result structure

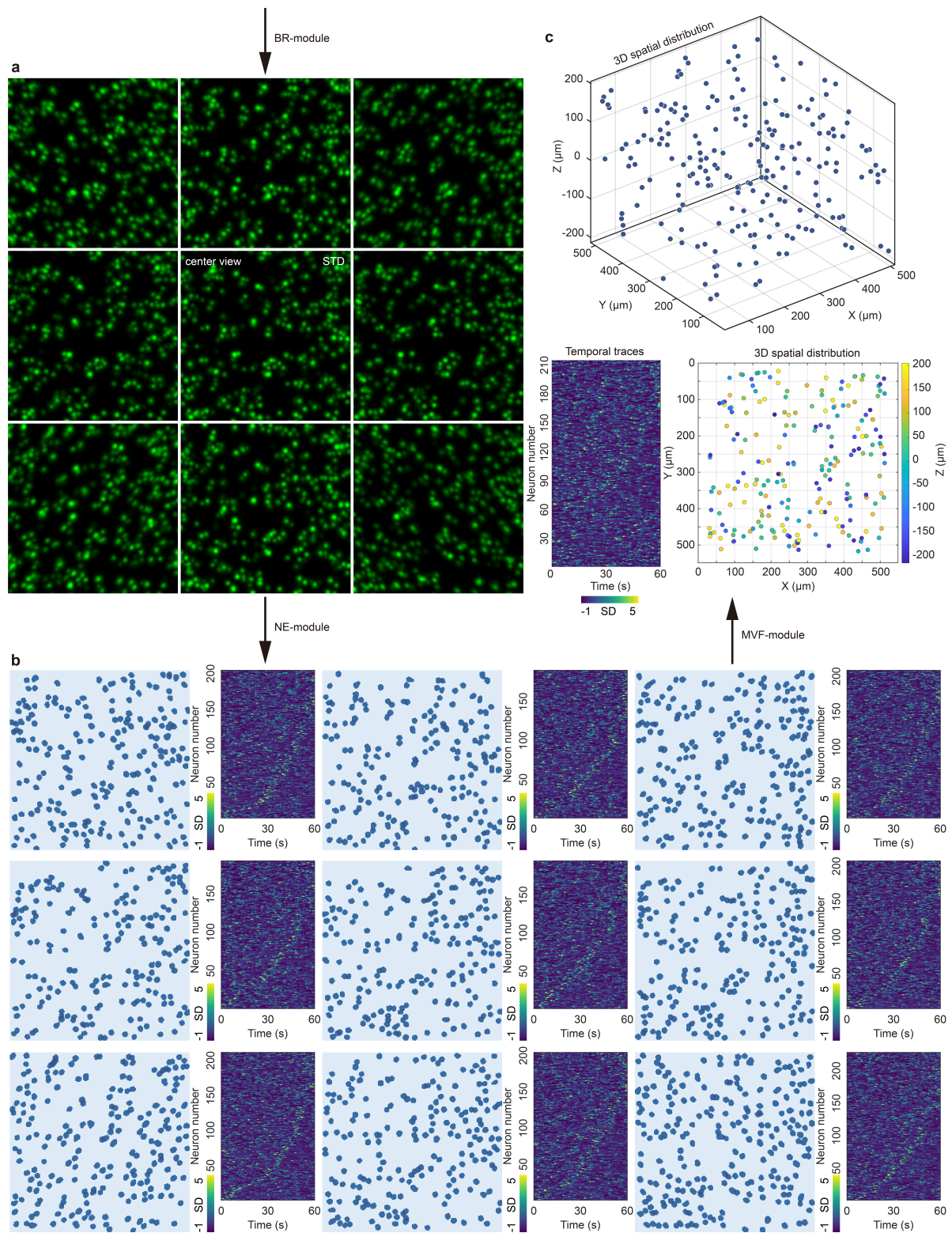
Upon completion, the `output_dir` will contain a structured sequence of results across seven distinct steps:

- `STEP_1_DENO`: Denoised data with enhanced SNR.
- `STEP_2_TR`: Data resampled to the target temporal resolution.
- `STEP_3_SR`: Data resampled to the target spatial resolution.
- `STEP_4_RMBG`: Background-subtracted data isolating neuronal signals.
- `STEP_5_SEG`: Short-term semantic masks identifying neuronal candidates.
- `STEP_6_MN`: Merged neuron instances and their extracted temporal traces.
- `STEP_7_VM`: **(3D Only)** Final 3D localization and merged temporal traces.
- `times`: Total running time of each processing step.

To provide a more intuitive understanding of the workflow, we use demo data to provide a step-by-step visual tutorial of DeepWonder3D. The following images demonstrate the outputs from the early stages of the DeepWonder3D pipeline for data preprocessing (i.e., denoising module, RR-module, and BR-module).



Below are the expected outputs from the late stages of the DeepWonder3D pipeline (i.e., NE-module and MVF-module).



The demo dataset provided in this repository is 977 MB. It comprises 13 views (view IDs correspond to [this figure](#)), with a resolution of 128×128 pixels, 1,200 frames, a 4 Hz frame rate, and a $4 \mu\text{m}$ pixel size. In this demo, we utilized the $2 \mu\text{m}$ pre-trained DeepWonder3D model for processing. To simulate a standard local workstation, we ran the demo on a 12th Gen Intel Core i7-12700 CPU (2.10 GHz) and a single NVIDIA GeForce RTX 3080 GPU (10 GB). Executing the demo with the following commands requires approximately 800 seconds, resulting in 15.3 GB of saved outputs:

```
conda activate dw3d
cd DeepWonder3D_pytorch
python main_pipeline_3d.py
```

The processing time for each step is detailed below:

- STEP_1_DENO: ~80s
- STEP_2_TR: ~70s
- STEP_3_SR: ~200s
- STEP_4_RMBG: ~200s
- STEP_5_SEG: ~90s
- STEP_6_MN: ~200s
- STEP_7_VM: ~7s

These durations include the input/output (I/O) time for reading the demo data and writing the 15.3 GB of results to disk. The processing time can be substantially reduced by utilizing higher-performance hardware or multi-GPU parallelization.

V. Model Training (main_train.py)

While pre-trained models are readily available in the `/pth` directory, `main_train.py` provides the flexibility to train or finetune the `SR`, `RMBG`, and `SEG` networks for specialized biological samples or specific microscope configurations.

Configure the `main_train_pipeline` in the `__main__` block of `main_train.py`:

- **Target selection:** Set `type='sr'`, `'rmbg'`, or `'seg'` to choose the model for training.
- **Data input:** Ensure `input_path` and `input_folder` point to your annotated training pairs. Training pair requirements for each processing step are detailed in the [figure below](#)
- **Sequential training:** You can combine keywords (e.g., `type='sr_rmbg'`) to train multiple architectures consecutively.

